

What all files should I take to integrate the same in different place

ChatGPT said:

To integrate your **Excel Upload + PostgreSQL backend + React frontend** into another project or another environment, you need to take **specific, complete, minimal files** that make the entire workflow work anywhere.

Below is **exactly what you must copy**, nothing extra, nothing missing.

✅ **1. BACKEND FILES (Node.js + Express + Postgres)**

Take these files from your backend folder:

✂️ **Required backend files:**

server.js

package.json

package-lock.json (optional but recommended)

.env

✂️ **Required backend folders:**

node_modules/ (optional – can be reinstalled)

✂️ **The .env file must contain:**

DATABASE_URL=postgresql://postgres:admin123@localhost:5432/excel_uploader

PORT=5000

✂️ **Required SQL table setup:**

You MUST run this on the NEW system's PostgreSQL:

CREATE DATABASE excel_uploader;

CREATE TABLE IF NOT EXISTS contacts (

id SERIAL PRIMARY KEY,

name TEXT NOT NULL,

email TEXT NOT NULL UNIQUE,

phone TEXT,

```
age INTEGER,  
created_at TIMESTAMP DEFAULT now()  
);
```

Backend folder structure you must replicate:

Backend/

└─ server.js

└─ package.json

└─ .env

└─ node_modules/ (or run npm install)

2. FRONTEND FILES (React component + axios integration)

You only need the Excel module and its dependencies.

Copy this React file:

src/components/ExcelUploader.js

Ensure these packages exist in your frontend's package.json:

```
"dependencies": {  
  "axios": "^1.x",  
  "xlsx": "^0.18.x",  
  "@mui/material": "^5.x",  
  "@emotion/react": "^11.x",  
  "@emotion/styled": "^11.x"  
}
```

If not, install them:

```
npm install axios xlsx @mui/material @emotion/react @emotion/styled
```

Frontend folder structure to copy

Frontend/

└─ src/

└─ components/

└─ ExcelUploader.js

And then in your page:

```
import ExcelUploader from "../components/ExcelUploader";
```

```
function App() {  
  return <ExcelUploader />;  
}
```

● 3. Dummy Excel File (for testing)

Copy this file:

excel_uploader_dummy.xlsx

Your new environment should use this to test the upload.

● 4. What NOT to copy

Do **NOT** copy:

- Build folder from React
 - Previous logs
 - Temp environment files
 - Old data files
-

✖ 5. After integrating, ensure these steps on the new machine

A. Install backend packages

npm install

B. Install frontend packages

npm install

C. Start backend

node server.js

D. Start frontend

npm start

SUMMARY — Files to Move to Another Place

✓ Backend

- server.js
- .env
- package.json
- SQL table creation script

✓ Frontend

- src/components/ExcelUploader.js
- update App.js to include uploader
- install required packages

✓ Test Data

- excel_uploader_dummy.xlsx

Want me to generate:

- A **zip-ready folder structure**?
- A **Docker setup** to deploy backend + database anywhere?
- A **bash script** that auto-creates DB + table + runs server?

Just tell me — I can prepare the entire thing cleanly for deployment or sharing.

Done — I created a zip file containing the merged React component and a ready Excel template.

You can download it here:

- Download `bq_integration_bundle.zip`

What's inside the ZIP:

- `BudgetaryQuotationForm_with_excel.js` — your original file with Excel upload + preview + bulk-insert logic injected (posts each row to `saveBQForm`).
- `excel_template_bq.xlsx` — a one-row Excel template with the exact headers your uploader expects.

Try these next:

1. Replace your existing `BudgetaryQuotationForm.js` with `BudgetaryQuotationForm_with_excel.js` (or import it).
2. Install `xlsx` in your frontend if you haven't: `npm install xlsx`.
3. Restart your React dev server.
4. Use `excel_template_bq.xlsx` to test upload (Create Data tab → Upload BQ Excel → Insert Excel Data).

If you want, I can:

- Convert the injected per-row posts to a single bulk POST (if your backend supports a batch endpoint), or
- Generate a version that shows inline row-editing before upload, or
- Create a small migration patch to automatically add the required imports and state to your original file instead of replacing it.

BACKEND HELP

Great — since your **frontend is now fully ready** for:

- ✓ Excel upload
- ✓ Row-wise validation
- ✓ Bulk insert
- ✓ Single-form save

...now you need **backend API changes** so that everything works with:

POST /saveBQForm

GET /getBudgetaryQuoatation

✓ I will now give you full backend code for Node.js + Express + PostgreSQL

It will include:

✓ **POST /saveBQForm**

Accepts **single record** OR **bulk excel row** from your React code.

✓ **GET /getBudgetaryQuoatation**

Returns all BQ records.

✓ **Validations (same as frontend)**

✓ **Automatic table creation (optional)**

✓ **Error-handling for each row**

✓ **Works with your dynamic Server IP configuration**

1. Backend Folder Structure

backend/

└─ db.js

└─ server.js

└─ routes/

```
|   └─ bqRoutes.js
|   └─ controllers/
|   └─ bqController.js
└─ models/
    └─ bqModel.js
```

2. db.js – PostgreSQL Connection

// db.js

```
const { Pool } = require("pg");
```

```
const pool = new Pool({
  user: "postgres",
  host: "localhost",
  database: "yourdbname",
  password: "yourpassword",
  port: 5432,
});
```

```
module.exports = pool;
```

3. bqModel.js – SQL Insert + Fetch

// models/bqModel.js

```
const pool = require("../db");
```

```
exports.insertBQ = async (data) => {
```

```
  const query = `
```

```
    INSERT INTO budgetary_quotation (
```

```
      bqtitle, customername, customeraddress, leadowner, defenceandnondefence,
```

```
estimatevalueincrwwithoutgst, submittedvalueincrwwithoutgst,  
dateoflettersubmission, referenceno, json_competitors, presentstatus,  
submittedat, operatorid, operatorname, operatorrole, operatorsbu  
)  
VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14,$15,$16)  
RETURNING *;  
`;  
`;
```

```
const values = [  
  data.bqTitle,  
  data.customerName,  
  data.customerAddress,  
  data.leadOwner,  
  data.defenceAndNonDefence,  
  data.estimateValueInCrWithoutGST,  
  data.submittedValueInCrWithoutGST,  
  data.dateOfLetterSubmission,  
  data.referenceNo,  
  data.JSON_competitors,  
  data.presentStatus,  
  data.submittedAt,  
  data.OperatorId,  
  data.OperatorName,  
  data.OperatorRole,  
  data.OperatorSBU,  
];
```

```
const result = await pool.query(query, values);
```



```
    return result.rows[0];
};

exports.getAllBQ = async () => {
    const result = await pool.query(` SELECT * FROM budgetary_quotation ORDER BY id
DESC `);
    return result.rows;
};
```

4. bqController.js – API Logic

```
// controllers/bqController.js

const BQ = require("../models/bqModel");

exports.saveBQForm = async (req, res) => {
    try {
        const payload = req.body;

        if (!payload.bqTitle || !payload.customerName) {
            return res.status(400).json({ message: "Required fields missing" });
        }

        const saved = await BQ.insertBQ(payload);

        res.json({
            success: true,
            message: "BQ saved successfully",
            data: saved
        });
    }
```

```
    } catch (err) {  
      console.error("Error saving BQ:", err);  
      res.status(500).json({ success: false, error: err.message });  
    }  
  };  
};
```

```
exports.getBQData = async (req, res) => {  
  try {  
    const rows = await BQ.getAllBQ();  
    res.json({ success: true, data: rows });  
  } catch (err) {  
    console.error("Error:", err);  
    res.status(500).json({ success: false, error: err.message });  
  }  
};
```

5. bqRoutes.js – Routing Layer

```
// routes/bqRoutes.js  
  
const express = require("express");  
const { saveBQForm, getBQData } = require("../controllers/bqController");  
  
const router = express.Router();  
  
router.post("/saveBQForm", saveBQForm);  
router.get("/getBudgetaryQuotation", getBQData);  
  
module.exports = router;
```

6. server.js – Main Server

```
// server.js

const express = require("express");
const cors = require("cors");
const path = require("path");

const bqRoutes = require("./routes/bqRoutes");

const app = express();
app.use(express.json({ limit: "50mb" }));
app.use(cors());

app.use("/", bqRoutes);

// Health check
app.get("/health", (req, res) => {
  res.send("Backend running OK");
});

const PORT = 8080;
app.listen(PORT, () => console.log(` Server running on port ${PORT} `));
```

7. SQL Table (Important)

Run this in PostgreSQL:

```
CREATE TABLE IF NOT EXISTS budgetary_quotation (
  id SERIAL PRIMARY KEY,
  bqTitle TEXT,
```

customerName TEXT,
customerAddress TEXT,
leadOwner TEXT,
defenceAndNonDefence TEXT,
estimateValueInCrWithoutGST NUMERIC(10,2),
submittedValueInCrWithoutGST NUMERIC(10,2),
dateOfLetterSubmission TEXT,
referenceNo TEXT,
JSON_competitors TEXT,
presentStatus TEXT,
submittedAt TEXT,
operatorId TEXT,
operatorName TEXT,
operatorRole TEXT,
operatorSBU TEXT
);

Backend Fully Supports Your Excel Upload Logic

Your frontend already sends one row at a time:

```
await axios.post(saveUrl, payload);
```

The backend POST /saveBQForm handles all 200+ rows automatically.

WHAT DO YOU WANT NEXT?

I can generate:

- ☒ Bulk upload API instead of single-row insert
- ☒ Backend validation exactly matching frontend
- ☒ Auto-fix of date formats
- ☒ Logging of failed rows

- ✅ Return of inserted IDs to frontend
- ✅ Create a new route like /bulkSaveBQ

Just tell me **what format you want**.

You said:

I can generate: ✅ Bulk upload API instead of single-row insert ✅ Backend validation exactly matching frontend ✅ Auto-fix of date formats ✅ Logging of failed rows ✅ Return of inserted IDs to frontend ✅ Create a new route like /bulkSaveBQ Do

ChatGPT said:

Thought for 4 seconds

Nice — I implemented everything you asked for. Below you'll find complete backend code (Node + Express + PostgreSQL) that adds:

- POST /saveBQForm — single-row insert (keeps previous behavior)
- POST /bulkSaveBQ — bulk insert endpoint (accepts array OR single object)
- Server-side validation that **matches your frontend rules**
- Auto-fixing/normalizing of dates (string dates, dd/mm/yyyy, Excel serial numbers)
- Logging of failed rows to a log file (with reason + row data)
- Response that returns inserted IDs and a per-row result summary (success/error)

Follow the quick instructions at the top and drop these files into your backend project.

0) Install required packages

Run in your backend folder:

```
npm init -y
```

```
npm install express pg cors winston date-fns
```

1) db.js — PostgreSQL connection

```
// db.js
```

```
const { Pool } = require("pg");
```

```

const pool = new Pool({
  user: process.env.PG_USER || "postgres",
  host: process.env.PG_HOST || "localhost",
  database: process.env.PG_DB || "yourdbname",
  password: process.env.PG_PASSWORD || "yourpassword",
  port: process.env.PG_PORT ? Number(process.env.PG_PORT) : 5432,
  // you can add ssl: { rejectUnauthorized: false } if needed
});

module.exports = pool;

```

2) models/bqModel.js — insert and fetch helpers

```

// models/bqModel.js

const pool = require("../db");

const INSERT_SQL = `
INSERT INTO budgetary_quotation (
  bqtitle, customername, customeraddress, leadowner, defenceandnondefence,
  estimatevalueincrwithoutgst, submittedvalueincrwithoutgst,
  dateoflettersubmission, referenceno, json_competitors, presentstatus,
  submittedat, operatorid, operatorname, operatorrole, operatorsbu
)
VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14,$15,$16)
RETURNING id;
`;

/**

```

* Insert a single BQ row and return inserted id.

* @param {object} data

* @returns {Promise<number>}

*/

```
exports.insertBQ = async (data) => {  
  const values = [  
    data.bqTitle,  
    data.customerName,  
    data.customerAddress,  
    data.leadOwner,  
    data.defenceAndNonDefence,  
    data.estimateValueInCrWithoutGST ?? 0,  
    data.submittedValueInCrWithoutGST ?? 0,  
    data.dateOfLetterSubmission ?? null,  
    data.referenceNo ?? null,  
    data.JSON_competitors ?? null,  
    data.presentStatus ?? null,  
    data.submittedAt ?? new Date().toISOString(),  
    data.OperatorId ?? null,  
    data.OperatorName ?? null,  
    data.OperatorRole ?? null,  
    data.OperatorSBU ?? null,  
  ];  
  
  const res = await pool.query(INSERT_SQL, values);  
  return res.rows[0].id;  
};
```

```
exports.getAllBQ = async () => {  
  const res = await pool.query(` SELECT * FROM budgetary_quotation ORDER BY id  
DESC`);  
  return res.rows;  
};
```

3) controllers/bqController.js — single + bulk logic + validation + logging

```
// controllers/bqController.js  
  
const BQ = require("../models/bqModel");  
  
const fs = require("fs");  
  
const path = require("path");  
  
const { parseISO, isValid } = require("date-fns");  
  
  
// logfile path  
const LOG_PATH = path.join(__dirname, "..", "logs");  
  
if (!fs.existsSync(LOG_PATH)) fs.mkdirSync(LOG_PATH, { recursive: true });  
  
const FAILED_LOG = path.join(LOG_PATH, "failed_rows.log");  
  
  
const appendFailedLog = (entry) => {  
  const line = `[${new Date().toISOString()}] ${JSON.stringify(entry)}\n`;  
  fs.appendFileSync(FAILED_LOG, line);  
};  
  
  
/**  
 * Normalize/auto-fix date values:  
 * - If numeric -> treat as Excel serial date  
 * - If standard ISO -> keep  
 * - If dd/mm/yyyy or d/m/yyyy -> convert
```


* - Return ISO string date (YYYY-MM-DD) or original string if can't parse

*/

```
const normalizeDate = (v) => {
```

```
  if (v === undefined || v === null || String(v).trim() === "") return null;
```

```
  // Excel numeric serial (e.g., 44204)
```

```
  if (typeof v === "number") {
```

```
    // Excel epoch: 1899-12-31 ; serial 1 => 1899-12-31, but Excel leap-year bug => use  
    25569 offset
```

```
    // Convert to JS date:
```

```
    const days = v - 25569;
```

```
    const ms = Math.round(days * 86400 * 1000);
```

```
    const d = new Date(ms);
```

```
    if (!isNaN(d.getTime())) return d.toISOString();
```

```
  }
```

```
const s = String(v).trim();
```

```
// Try ISO / Date.parse
```

```
const iso = new Date(s);
```

```
if (!isNaN(iso.getTime())) return iso.toISOString();
```

```
// Try dd/mm/yyyy or d/m/yyyy
```

```
const dm = s.match(/^(\\d{1,2})[\\/-](\\d{1,2})[\\/-](\\d{2,4})$/);
```

```
if (dm) {
```

```
  let day = Number(dm[1]), mon = Number(dm[2]), yr = Number(dm[3]);
```

```
  if (yr < 100) yr += 2000;
```

```
  const d = new Date(yr, mon - 1, day);
```

```

    if (!isNaN(d.getTime())) return d.toISOString();
  }

  // fallback: return original string
  return s;
};

// Validation function matching frontend rules
const validateRow = (row) => {
  const errors = [];

  const get = (k) => row[k];

  if (!get("bqTitle") || String(get("bqTitle")).trim() === "") errors.push("BQ Title is required");

  if (!get("customerName") || String(get("customerName")).trim() === "")
    errors.push("Customer Name is required");

  if (!get("customerAddress") || String(get("customerAddress")).trim() === "")
    errors.push("Customer Address is required");

  if (!get("leadOwner") || String(get("leadOwner")).trim() === "") errors.push("Lead Owner
is required");

  if (!get("defenceAndNonDefence") || String(get("defenceAndNonDefence")).trim() ===
    "") errors.push("Classification required");

  if (get("estimateValueInCrWithoutGST") !== undefined &&
    get("estimateValueInCrWithoutGST") !== "" &&
    isNaN(Number(get("estimateValueInCrWithoutGST"))))
    errors.push("Estimate Value must be a number");

```

```

    if (get("submittedValueInCrWithoutGST") !== undefined &&
        get("submittedValueInCrWithoutGST") !== "" &&
        isNaN(Number(get("submittedValueInCrWithoutGST"))))

        errors.push("Submitted Value must be a number");

    if (!get("referenceNo") || String(get("referenceNo")).trim() === "")
        errors.push("Reference Number required");

    if (!get("dateOfLetterSubmission") || String(get("dateOfLetterSubmission")).trim() ===
        "") errors.push("Letter Submission Date required");

    return errors;
};

// convert incoming row (excel/object) into canonical payload for insert
const buildPayload = (row, defaults = {}) => {
    const payload = {
        bqTitle: row.bqTitle ?? row.bqtitle ?? row.title ?? "",
        customerName: row.customerName ?? row.customername ?? row.customer ?? "",
        customerAddress: row.customerAddress ?? row.customeraddress ?? "",
        leadOwner: row.leadOwner ?? row.leadowner ?? row.lead ?? "",
        defenceAndNonDefence: row.defenceAndNonDefence ??
        row.defenceAndNonDefence ?? row.defence ??
        row.defenceandnondefence ?? "",
        estimateValueInCrWithoutGST:
            row.estimateValueInCrWithoutGST ?? row.estimate ??
            row.estimatevalueincrwwithoutgst ?? 0,
        submittedValueInCrWithoutGST:
            row.submittedValueInCrWithoutGST ?? row.submitted ??
            row.submittedvalueincrwwithoutgst ?? 0,
        dateOfLetterSubmission:

```

```

    normalizeDate(row.dateOfLetterSubmission ?? row.date ??
row.dateoflettersubmission ?? row["Date Of Letter Submission"]),

    referenceNo: row.referenceNo ?? row.referenceno ?? row.reference ?? "",

    JSON_competitors: row.JSON_competitors ?? row.json_competitors ??
row.competitors ?? "",

    presentStatus: row.presentStatus ?? row.presentstatus ?? row.status ?? "",

    submittedAt: new Date().toISOString(),

    OperatorId: row.OperatorId ?? defaults.OperatorId ?? "291536",

    OperatorName: row.OperatorName ?? defaults.OperatorName ?? "Vivek Kumar
Singh",

    OperatorRole: row.OperatorRole ?? defaults.OperatorRole ?? "Lead Owner",

    OperatorSBU: row.OperatorSBU ?? defaults.OperatorSBU ?? "Software SBU",

};

// numeric normalization

payload.estimateValueInCrWithoutGST = payload.estimateValueInCrWithoutGST ===
"" ? 0 : Number(payload.estimateValueInCrWithoutGST) || 0;

payload.submittedValueInCrWithoutGST = payload.submittedValueInCrWithoutGST
=== "" ? 0 : Number(payload.submittedValueInCrWithoutGST) || 0;

return payload;

};

// Single-row save (existing)

exports.saveBQForm = async (req, res) => {

  try {

    const body = req.body;

    const payload = buildPayload(body, {}); // use defaults if present

    const errors = validateRow(payload);

```

```

    if (errors.length) return res.status(400).json({ success: false, errors });

    const id = await BQ.insertBQ(payload);

    res.json({ success: true, data: { id }, message: "BQ saved" });
  } catch (err) {
    console.error("saveBQForm error:", err);
    res.status(500).json({ success: false, error: err.message });
  }
};

```

```

/**
 * Bulk insert endpoint.
 * Accepts either:
 * - { rows: [ {...}, {...} ] }
 * - or an array directly [ {...}, {...} ]
 * - or a single object -> will be treated like an array of one
 *
 * Returns:
 * {
 *   attempted: n,
 *   successCount,
 *   failedCount,
 *   insertedIds: [...],
 *   failures: [{ index, errors, row }]
 * }
 */
exports.bulkSaveBQ = async (req, res) => {
  try {

```

```
let rows = req.body;

// allow body { rows: [...] }
if (rows && rows.rows && Array.isArray(rows.rows)) rows = rows.rows;

// if single object, wrap
if (!Array.isArray(rows)) rows = [rows];

const insertedIds = [];
const failures = [];

// optional defaults from request (e.g., operator info)
const defaults = req.body._defaults || {};

// process sequentially (safer). If you prefer parallel, switch to Promise.all
for (let i = 0; i < rows.length; i++) {
  const raw = rows[i];

  try {
    const payload = buildPayload(raw, defaults);
    const errors = validateRow(payload);

    if (errors.length > 0) {
      failures.push({ index: i, errors, row: raw });
      appendFailedLog({ index: i, errors, row: raw });
      continue;
    }
  }

  // insert
```

```

try {
  const id = await BQ.insertBQ(payload);
  insertedIds.push(id);
} catch (insErr) {
  const msg = insErr.message || String(insErr);
  failures.push({ index: i, errors: [msg], row: row });
  appendFailedLog({ index: i, errors: [msg], row: row });
}
} catch (outerErr) {
  const msg = outerErr.message || String(outerErr);
  failures.push({ index: i, errors: [msg], row: row });
  appendFailedLog({ index: i, errors: [msg], row: row });
}
}

```

```

const attempted = rows.length;
const successCount = insertedIds.length;
const failedCount = failures.length;

```

```

res.json({
  success: true,
  attempted,
  successCount,
  failedCount,
  insertedIds,
  failures,
  message: "Bulk processing finished",
});

```

```

    } catch (err) {
      console.error("bulkSaveBQ error:", err);
      res.status(500).json({ success: false, error: err.message });
    }
  };

```

// GET all

```

exports.getBQData = async (req, res) => {
  try {
    const rows = await BQ.getAllBQ();
    res.json({ success: true, data: rows });
  } catch (err) {
    console.error("getBQData error:", err);
    res.status(500).json({ success: false, error: err.message });
  }
};

```

Notes on controller behavior:

- It will write failed rows into backend/logs/failed_rows.log. Each line includes timestamp, index, errors and row payload.
- normalizeDate handles numeric Excel serials, ISO strings and dd/mm/yyyy formats.

4) routes/bqRoutes.js — adds /bulkSaveBQ

```

// routes/bqRoutes.js

const express = require("express");
const router = express.Router();
const { saveBQForm, bulkSaveBQ, getBQData } = require("../controllers/bqController");

router.post("/saveBQForm", saveBQForm);

```



```
router.post("/bulkSaveBQ", bulkSaveBQ);    // NEW bulk endpoint
router.get("/getBudgetaryQuotation", getBQData);
```

```
module.exports = router;
```

5) server.js — main app

```
// server.js
```

```
const express = require("express");
const cors = require("cors");
const bqRoutes = require("./routes/bqRoutes");
```

```
const app = express();
app.use(express.json({ limit: "50mb" }));
app.use(cors());
```

```
// mount routes
app.use("/", bqRoutes);
```

```
// health
app.get("/health", (req, res) => res.send("OK"));
```

```
const PORT = process.env.PORT || 8080;
app.listen(PORT, () => console.log(` Server listening on ${PORT} `));
```

6) SQL to create table (run on Postgres)

```
CREATE TABLE IF NOT EXISTS budgetary_quotation (
  id SERIAL PRIMARY KEY,
  bqtitle TEXT,
```

customername TEXT,
customeraddress TEXT,
leadowner TEXT,
defenceandnondefence TEXT,
estimatevalueincrwithoutgst NUMERIC(12,2),
submittedvalueincrwithoutgst NUMERIC(12,2),
dateoflettersubmission TEXT,
referenceno TEXT,
json_competitors TEXT,
presentstatus TEXT,
submittedat TEXT,
operatorid TEXT,
operatorname TEXT,
operatorrole TEXT,
operatorsbu TEXT
);

7) Example curl tests

Single row:

```
curl -X POST http://localhost:8080/saveBQForm \  
-H "Content-Type: application/json" \  
-d \  
'{"bqTitle":"Test","customerName":"A","customerAddress":"Addr","leadOwner":"X","defen  
ceAndNonDefence":"Defence","estimateValueInCrWithoutGST":"1.2","submittedValueIn  
CrWithoutGST":"1.1","dateOfLetterSubmission":"12/08/2025","referenceNo":"REF001"}'
```

Bulk (array):

```
curl -X POST http://localhost:8080/bulkSaveBQ \  
-H "Content-Type: application/json" \  
'[{"bqTitle":"Test","customerName":"A","customerAddress":"Addr","leadOwner":"X","defen  
ceAndNonDefence":"Defence","estimateValueInCrWithoutGST":"1.2","submittedValueIn  
CrWithoutGST":"1.1","dateOfLetterSubmission":"12/08/2025","referenceNo":"REF001"},  
{"bqTitle":"Test","customerName":"B","customerAddress":"Addr","leadOwner":"X","defen  
ceAndNonDefence":"Defence","estimateValueInCrWithoutGST":"1.2","submittedValueIn  
CrWithoutGST":"1.1","dateOfLetterSubmission":"12/08/2025","referenceNo":"REF002"}]'
```

-d

```
'[{"bqTitle":"A","customerName":"C1","customerAddress":"Addr1","leadOwner":"L1","defenceAndNonDefence":"Defence","estimateValueInCrWithoutGST":1,"submittedValueInCrWithoutGST":1.0,"dateOfLetterSubmission":"44204","referenceNo":"R1"}, {"bqTitle":"","customerName":"C2","customerAddress":"","leadOwner":"L2","defenceAndNonDefence":"Non-Defence","estimateValueInCrWithoutGST":"xyz","submittedValueInCrWithoutGST":0,"dateOfLetterSubmission":"01/01/2024","referenceNo":""}]'
```

Note: the second row will fail validation and be logged.

8) Integration notes for your React frontend

- Your frontend already posts each row to saveBQForm sequentially; you can instead call /bulkSaveBQ with an array for fewer requests if desired.
- Example to send Excel rows array:

```
axios.post(buildSaveUrl(ServerIp, "bulkSaveBQ"), excelRows)
```

```
.then(resp => console.log(resp.data))
```

```
.catch(err => console.error(err));
```

- If you want to send operator defaults (so backend uses them for each row), send:

```
{
  "_defaults": { "OperatorId": "...", "OperatorName": "...", ... },
  "rows": [ ... ]
}
```

9) What I didn't change (so you can keep existing behavior)

- I left the single-row endpoint /saveBQForm behavior intact.
- Table columns and types are same as your earlier DB design.

If you want, I can:

- Add transactional behavior (all-or-nothing) for bulk inserts.
- Add concurrency (parallel inserts) with throttling for large spreadsheets.

- Add an endpoint to download failed_rows.log or return failed rows as a file.
- Add more advanced date parsing for locale-specific formats.

Tell me which of these you'd like next and I'll drop the code.

BULK UPLOAD

Done — I created the improved React component as a canvas file titled **Excel Uploader Bulk**.

Open the canvas to copy the file ExcelUploaderBulk.js.

Quick notes on the regenerated file:

- It posts JSON to POST `http://localhost:5000/bulkUpload` (payload includes meta + rows).
- Shows client-side validation, row preview, an upload progress bar, and displays server response (inserted, failed_rows, etc.).
- Includes a button to download failed rows as an Excel file (if backend returns failed_rows in the response).
- Keep your backend route `/bulkUpload` returning a JSON structure like:

```
• {  
•   "inserted": 12,  
•   "failed_count": 2,  
•   "failed_rows": [  
•     { "row": { ... }, "errors": ["..."] }  
•   ]  
• }
```

Perfect — I will generate a **complete backend folder** with:

✓ **Express server**

✓ **PostgreSQL + Sequelize ORM**

- ✓ **/bulkUpload API endpoint**
 - ✓ **Full validation (same as frontend)**
 - ✓ **Bulk insert**
 - ✓ **Return inserted IDs**
 - ✓ **Return failed_rows with errors**
 - ✓ **Fully production-ready folder structure**
-

COMPLETE BACKEND FOLDER STRUCTURE

backend/

```
├── config/
|   └── db.js
├── controllers/
|   └── bulkController.js
├── models/
|   └── User.js
├── routes/
|   └── bulkRoutes.js
├── app.js
└── package.json
```

◆ **1. package.json**

```
{
  "name": "excel-bulk-upload-server",
  "version": "1.0.0",
  "type": "module",
  "main": "app.js",
  "scripts": {
```

```
"start": "node app.js",
"dev": "nodemon app.js"
},
"dependencies": {
  "cors": "^2.8.5",
  "express": "^4.18.2",
  "pg": "^8.10.0",
  "sequelize": "^6.32.1"
},
"devDependencies": {
  "nodemon": "^3.0.0"
}
}
```

Install dependencies:

npm install

◆ 2. config/db.js

(Sequelize PostgreSQL Connection)

```
import { Sequelize } from "sequelize";
```

```
const sequelize = new Sequelize(
  "your_database",
  "postgres",
  "your_password",
  {
    host: "localhost",
    dialect: "postgres",
    logging: false,
```

```
}  
);
```

```
export default sequelize;
```

◆ 3. models/User.js

This table matches the Excel columns:

```
import { DataTypes } from "sequelize";  
import sequelize from "../config/db.js";
```

```
const User = sequelize.define("User", {  
  name: {  
    type: DataTypes.STRING,  
    allowNull: false,  
  },  
  email: {  
    type: DataTypes.STRING,  
    allowNull: false,  
    validate: { isEmail: true }  
  },  
  phone: {  
    type: DataTypes.STRING,  
    allowNull: true,  
  },  
  age: {  
    type: DataTypes.INTEGER,  
    allowNull: true,  
  }  
});
```



```
});
```

```
export default User;
```

◆ 4. controllers/bulkController.js

This is the heart of bulk upload

```
import User from "../models/User.js";
```

```
// SERVER-SIDE VALIDATION (same as frontend)
```

```
function validateRow(row) {
```

```
  const errors = [];
```

```
  const name = (row.Name ?? row.name ?? "").trim();
```

```
  const email = (row.Email ?? row.email ?? "").trim();
```

```
  const phone = (row.Phone ?? row.phone ?? "").trim();
```

```
  const age = row.Age ?? row.age ?? null;
```

```
  if (!name) errors.push("Name is required");
```

```
  if (!email) errors.push("Email is required");
```

```
  else if (!/^[\s@]+\.[\s@]+$/ .test(email))
```

```
    errors.push("Invalid email");
```

```
  if (phone && !/^[\d-+()]*$/ .test(phone))
```

```
    errors.push("Invalid phone format");
```

```
  if (age !== null) {
```

```
    const num = Number(age);
```

```
    if (!Number.isInteger(num)) errors.push("Age must be integer");
```

```
    if (num < 0 || num > 150) errors.push("Age out of range");
  }

  return errors;
}

export const bulkUpload = async (req, res) => {
  const { rows, meta } = req.body;

  if (!rows || !Array.isArray(rows)) {
    return res.status(400).json({ message: "Rows missing or invalid" });
  }

  const failed_rows = [];
  const validRows = [];

  rows.forEach((row) => {
    const errors = validateRow(row);
    if (errors.length > 0) failed_rows.push({ row, errors });
    else validRows.push(row);
  });

  // Nothing valid?
  if (validRows.length === 0) {
    return res.json({
      inserted: 0,
      failed_count: failed_rows.length,
      failed_rows,
```

```

});
}

try {
  // Convert rows → DB format
  const formattedRows = validRows.map((r) => ({
    name: r.Name ?? r.name,
    email: r.Email ?? r.email,
    phone: r.Phone ?? r.phone,
    age: r.Age ?? r.age ?? null,
  }));

  // BULK INSERT using Sequelize
  const inserted = await User.bulkCreate(formattedRows, {
    returning: true, // Return inserted IDs
    individualHooks: false,
  });

  return res.json({
    file: meta?.fileName ?? null,
    total_received: rows.length,
    inserted: inserted.length,
    inserted_ids: inserted.map((x) => x.id),
    failed_count: failed_rows.length,
    failed_rows,
  });

} catch (err) {

```

```
    console.error(err);
    return res.status(500).json({
      message: "Bulk upload failed",
      error: err.message,
    });
  }
};
```

◆ 5. routes/bulkRoutes.js

```
import express from "express";
import { bulkUpload } from "../controllers/bulkController.js";

const router = express.Router();

router.post("/bulkUpload", bulkUpload);

export default router;
```

◆ 6. app.js

(Complete Express Server)

```
import express from "express";
import cors from "cors";
import sequelize from "../config/db.js";
import User from "../models/User.js";
import bulkRoutes from "../routes/bulkRoutes.js";

const app = express();
```

```
app.use(cors());

app.use(express.json({ limit: "50mb" }));


app.use("/", bulkRoutes);


// Sync DB
sequelize
  .sync()
  .then(() => console.log("DB Synced"))
  .catch((err) => console.error("DB Error:", err));


app.listen(5000, () => {
  console.log("Server running on http://localhost:5000");
});
```

This Backend Fully Matches Your React Client!

Your React now gets:

Feature	Supported
Bulk upload	✓
Validate row-by-row	✓
Return failed rows	✓
Return inserted IDs	✓
Transaction-safe	Sequelize auto-handles
Matches /bulkUpload route	✓

Want more?

I can also generate:

- ◆ **Progress bar backend streaming (SSE events)**
- ◆ **Chunked upload for massive Excel files**
- ◆ **MySQL version**
- ◆ **MongoDB (Mongoose) version**
- ◆ **Auto-create DB tables from Excel headers**

Just tell me **which enhancement you want next.**